The Complete

Claude Code

Guide for Beginners

Your step-by-step companion to AI-powered development

FREE STUDENT RESOURCE

18 Chapters + 3 Appendices • Beginner Friendly • Zero Coding Experience Required

by AI Agents Accelerator

Table of Contents

Chapter 1

What Is Claude Code & Why It Matters

AI pair programming explained — what it can and can't do

Claude Code is a tool made by Anthropic that lives in your terminal — the text-based window where developers type commands. Think of it as having an experienced programmer sitting next to you, reading your project files, suggesting changes, writing code, and even running commands — all through a simple conversation.

Unlike chatbots you might have used (like ChatGPT or Claude.ai in a browser), Claude Code doesn't just answer questions. It actually sees your files, edits your code, runs terminal commands, and manages your git history. It works inside your project, not outside of it.

What can Claude Code do?

- Write code — from a single function to an entire application
- Read and understand your existing codebase, no matter how large
- Edit files — make changes, refactor, rename variables across your project
- Run commands — install packages, start servers, run tests
- Debug errors — paste an error message and let Claude trace the cause
- Manage git — commit, push, create branches, write pull request descriptions
- Connect to tools — via MCP servers (GitHub, Slack, databases, and more)

What Claude Code is NOT

Claude Code is not a magic wand. It doesn't replace the need to understand what you're building. Think of it as a very fast, very knowledgeable assistant: you decide what to build, Claude helps you figure out how. You're the architect, Claude is the builder.

Tip
You don't need to know how to code to use Claude Code. But you'll learn faster if you try to understand what Claude writes for you, rather than just accepting every suggestion blindly.

Who is this guide for?

This guide is for complete beginners. If you've never opened a terminal before, if you don't know what "git" means, if "API" sounds like alphabet soup — you're in the right place. Every concept is explained from scratch, with examples you can copy and try immediately.

How to use this guide

Read it in order. Each chapter builds on the previous one. Every chapter has the same structure: a short introduction explaining the concept, practical examples with real prompts you can copy, a "Try This" exercise to practice, and Key Takeaways summarizing what you learned.

Try This

Before reading further, go to `claude.ai` and make sure you have a paid account (Pro at $20/month or Max at $100–200/month). Claude Code requires a paid plan — the free tier doesn't include terminal access.

Key Takeaways

- ✓ Claude Code is an AI assistant that lives in your terminal and works inside your project
- ✓ It can read, write, edit files, run commands, and manage git
- ✓ You're the decision-maker — Claude is the fast, smart helper
- ✓ This guide assumes zero prior coding experience
- ✓ You need a paid Anthropic account (Pro $20/mo or Max $100–200/mo)

Chapter 2

Installation & First Launch

Mac + Windows step by step, authentication, your first conversation

---

Installing Claude Code takes about 2 minutes. There's a single command to run, and then you log in through your browser. That's it.

What you need before starting

- A computer — Mac, Windows, or Linux
- Internet connection — Claude Code talks to Anthropic's servers
- A paid Anthropic account — Claude Pro ($20/mo) or Max ($100–200/mo)

Note
You do not need Node.js anymore. The native installer has zero dependencies. The old `npm install` method still works but is no longer recommended.

Installation on Mac / Linux

Open Terminal (on Mac: press Cmd + Space, type "Terminal", press Enter). Then run:

```
curl -fsSL https://claude.ai/install.sh | bash
```

That's it. The installer downloads Claude Code, puts it in the right place, and sets up automatic updates.

Installation on Windows

First, install Git for Windows from `git-scm.com` if you don't have it. Claude Code needs it to run commands internally.

Then open PowerShell and run:

```
irm https://claude.ai/install.ps1 | iex
```

Verify it worked

In your terminal, type:

```
claude --version
```

You should see a version number (like `2.x.x`). If you see an error, see the Troubleshooting section in Chapter 15.

First launch & authentication

Navigate to any folder on your computer and type `claude`. On the first run, it will open your browser and ask you to log in with your Anthropic account. After you approve, a session token is saved locally — you won't need to log in again.

```
cd ~/Desktop
claude
```

You'll see a welcome message and a prompt where you can start typing. Congratulations — Claude Code is ready.

Tip
If you see "command not found", close your terminal and open a new one. The installer adds Claude to your PATH, but the change only takes effect in new terminal windows.

Alternative installation methods

If the native installer doesn't work for your setup, there are alternatives:

- Homebrew (Mac): `brew install --cask claude-code`
- npm (any platform): `npm install -g @anthropic-ai/claude-code` (requires Node.js 18+)
- Desktop App: Download from `claude.ai` for a graphical interface

Warning
Never use `sudo` with npm install. It causes permission issues and security risks. If you get permission errors, use `nvm` to install Node.js in your home directory instead.

Try This
Install Claude Code on your machine right now. Run `claude --version` to confirm it works. Then run `claude` in any folder and say "Hi! What can you do?" to start your first conversation.

Key Takeaways

- ✓ Native installer: one command, no dependencies, auto-updates
- ✓ Mac/Linux: curl command in Terminal
- ✓ Windows: PowerShell command + Git for Windows required
- ✓ First run opens browser for one-time authentication
- ✓ If "command not found" — open a new terminal window

Chapter 3

Your First Project

Create a folder, start Claude, build something simple together

---

The best way to learn Claude Code is by doing. In this chapter, you'll create a simple project from scratch and see Claude Code in action.

Step 1: Create a project folder

```
mkdir my-first-project
cd my-first-project
```

`mkdir` creates a new folder. `cd` moves you into it. Now you're "inside" your project.

Step 2: Start Claude Code

```
claude
```

Claude scans the folder (it's empty right now) and waits for your instructions.

Step 3: Build something!

Try this prompt:

```
Create a simple HTML page that shows a personal portfolio.
Include my name "Alex", a short bio, and 3 project cards.
Make it look modern with CSS. Use a dark theme.
```

Watch what happens. Claude will create an `index.html` file with all the HTML and CSS, explain what it built, and ask if you want any changes. It might also suggest creating additional files.

Step 4: View your creation

Open the file in your browser. On Mac: `open index.html`. On Windows: `start index.html`. Or just double-click the file in your file explorer.

Step 5: Make changes through conversation

Now try iterating. Say things like:

```
Change the color scheme to blue and white

Add a contact form at the bottom

Make it responsive so it looks good on phones
```

Each time, Claude will edit the file and show you exactly what changed. You don't need to know HTML or CSS — just describe what you want in plain English.

Tip

Be specific. “Make it better” is vague. “Make the headings larger, add more spacing between sections, and change the background to navy blue” gives Claude something concrete to work with.

Try This

Build your own portfolio page. Start with the prompt above, then make at least 3 changes through conversation. Notice how Claude remembers the context of your project and builds on it.

Key Takeaways

- ✓ Always start Claude Code from inside your project folder
- ✓ Describe what you want in plain English — be specific
- ✓ Claude creates and edits files directly in your project
- ✓ You can iterate endlessly through conversation
- ✓ You don’t need to know any programming language to get started

Chapter 4

Talking to Claude Code

Prompt engineering for beginners — the WHAT/WHERE/HOW/VERIFY formula

---

The quality of Claude's output depends almost entirely on the quality of your prompt. A good prompt isn't about using fancy words — it's about being clear and specific.

The WHAT / WHERE / HOW / VERIFY formula

Every good Claude Code prompt answers four questions:

WHAT — What do you want Claude to do?
Example: "Add a search bar to the homepage"

WHERE — Where in the project should it work?
Example: "In src/components/Header.jsx"

HOW — Any specific requirements or constraints?
Example: "Use the existing CSS variables, no new libraries"

VERIFY — How should Claude verify it works?
Example: "Run the dev server and check it renders"

Bad vs good prompts

Bad: `"Fix the bug"`

Claude doesn't know which bug, where, or what the expected behavior is.

Good: `"The login form in src/Login.jsx crashes when I submit empty fields. It should show a validation error instead. Check that both email and password fields are validated before submit."`

Prompt patterns that work well

- Explain before acting: `"Explain what this file does, then refactor it to be more readable"`
- Step-by-step: `"Walk me through setting up a database connection, one step at a time"`
- Constrained output: `"Write a function that does X. Keep it under 20 lines. No external libraries."`
- Review mode: `"Review this code for bugs, security issues, and performance problems"`

Tip

You can use ultrathink at the start of a complex prompt to make Claude think more deeply before responding. Try: “ultrathink: design a database schema for a todo app with user accounts, sharing, and labels.”

Try This

Open Claude Code in your project. Try asking the same question two ways: first vaguely (“make the code better”), then specifically using the WHAT/WHERE/HOW/VERIFY formula. Compare the results.

Key Takeaways

- ✓ Use the WHAT/WHERE/HOW/VERIFY formula for every complex prompt
- ✓ Specific prompts get dramatically better results than vague ones
- ✓ Prefix complex prompts with “ultrathink” for deeper analysis
- ✓ Tell Claude to explain before acting when you want to understand the changes
- ✓ Iterate — your first prompt doesn’t have to be perfect

Chapter 5

Working with Files

Reading, editing, creating files — the @file shortcut

Files are the core of any project. Claude Code can read, create, edit, and delete files in your project. Here's how to work with them effectively.

Reading files

Claude automatically scans your project structure. But you can point it to specific files:

```
What does src/App.jsx do? Explain it simply.

Read package.json and tell me what libraries this project uses.
```

The @file shortcut

Use @ followed by a filename to explicitly include a file in your prompt's context:

```
@src/App.jsx Refactor this component to use hooks instead of class syntax
```

This is useful when your project has many files and you want Claude to focus on specific ones.

Creating files

Just ask Claude to create what you need:

```
Create a .gitignore file with common Node.js exclusions

Create a README.md for this project with installation instructions

Create a new component called UserProfile in src/components/
```

Editing files

Claude shows you a diff (red for removed, green for added) before making changes. You approve or reject each change:

```
In src/utils/api.js, add error handling to the fetch calls

Rename all instances of "userName" to "username" across the project
```

Multi-file operations

Claude can work across multiple files at once:

```
Move the Header component from App.jsx into its own file
at src/components/Header.jsx and update all imports
```

Tip

If Claude makes a change you don’t like, press Esc Esc (Escape twice) to rewind the last action. This is your undo button.

Try This

In your project, ask Claude to: (1) create a new file, (2) read an existing file and explain it, (3) edit a file with a specific change. Practice approving and rejecting diffs.

Key Takeaways

- ✓ Use @filename to point Claude to specific files
- ✓ Claude shows diffs before making changes — you approve each one
- ✓ Esc Esc (double Escape) rewinds the last action
- ✓ Claude can create, read, edit, and delete files
- ✓ Multi-file operations work naturally through conversation

Chapter 6

Commands & Shortcuts

Essential slash commands, keyboard shortcuts, and modes

Claude Code has built-in commands (starting with `/`) and keyboard shortcuts that make you faster. You don't need to memorize all of them — start with the essentials and learn more as you go.

Essential slash commands

- **`/help`** — Show all available commands
- **`/status`** — Check your connection, model, and session info
- **`/compact`** — Compress conversation to save context (more on this in Ch. 7)
- **`/clear`** — Clear conversation history completely
- **`/cost`** — Show how much this session has cost so far
- **`/config`** — Open Claude Code settings
- **`/bug`** — Report a bug to Anthropic
- **`/doctor`** — Auto-diagnose common problems

Keyboard shortcuts

- **`Esc Esc`** — Rewind/undo the last Claude action
- **`Tab`** — Toggle extended thinking on/off
- **`Ctrl+R`** — Search your prompt history
- **`# message`** — Add a quick note to your CLAUDE.md file
- **`Ctrl+C`** — Cancel the current Claude response

The -p flag (non-interactive mode)

For quick one-off tasks, use `-p` to run a prompt without entering interactive mode:

```
claude -p "What framework does this project use?"

claude -p "Count the number of TODO comments in this project"
```

This runs the task, prints the result, and exits — great for quick questions.

> **Tip**
> Use `/cost` regularly to keep track of how much you're spending. Claude Code uses API tokens, and complex tasks can add up. More on cost management in Chapter 15.

Try This

Start a Claude Code session and try: `/status`, `/help`, and `/cost`. Then exit and try the `-p` flag with a quick question about your project.

Key Takeaways

- ✓ Slash commands start with / and control Claude Code itself
- ✓ /status, /compact, /clear, and /cost are the most useful daily commands
- ✓ Esc Esc rewinds mistakes, Tab toggles thinking, Ctrl+R searches history
- ✓ Use -p for quick one-off questions without entering interactive mode
- ✓ Use /doctor when something feels broken

Chapter 7

Context Management

Keep Claude focused and effective over long sessions

Claude Code has a context window — the amount of information it can "hold in its head" at once. As your conversation grows, older messages get pushed out. Managing this context is one of the most important skills to learn.

Why context matters

Imagine you're writing notes on a whiteboard. The whiteboard has limited space. As you write more, older notes get erased. That's what happens with Claude's context. If you've been chatting for 30+ minutes about many different topics, Claude may "forget" things from the beginning of your conversation.

How to manage context

/compact — Compress your conversation

When your conversation gets long, run `/compact`. Claude will summarize everything so far into a condensed version, freeing up space for new work. It's like taking notes and then cleaning up the whiteboard.

```
/compact
```

/clear — Start fresh

If you're switching to a completely different task, `/clear` wipes the conversation entirely. Use it when context from a previous task would only confuse Claude.

/status — Check your usage

Run `/status` to see how much of your context window is used. If it's getting high, it's time to `/compact`.

Best practices

- One task per session. Don't mix "build the login page" with "fix the database schema"
- Use `/compact` every 15–20 minutes in long sessions
- If Claude seems confused or forgets something, `/compact` and re-state the key facts
- Start new sessions for unrelated tasks rather than continuing a stale one

Try This

Start a session, work on something for 10+ prompts, then run `/compact`. Notice how Claude summarizes your conversation. Then ask a follow-up question — does it still remember the important context?

Key Takeaways

- ✓ Claude has a limited context window — think of it as a whiteboard
- ✓ /compact compresses your conversation without losing important context
- ✓ /clear wipes everything — use for completely new tasks
- ✓ One task per session keeps Claude focused and effective
- ✓ Run /compact every 15–20 minutes in long sessions

Chapter 8

Debugging, Testing & Fixing Errors

Paste the error, describe the symptom, let Claude trace it — then write tests to prevent it

Errors happen. A lot. Even experienced developers spend much of their time debugging. Claude Code is incredibly good at this — often better than searching Stack Overflow for hours.

The debugging workflow

Step 1: Copy the error message. All of it.

Step 2: Paste it to Claude and describe what you were doing:

```
I ran npm start and got this error:

[paste the full error here]

I was trying to add a new route to the Express server.
It worked before I added the /users endpoint.
```

Step 3: Let Claude trace the cause. It will read the relevant files, find the problem, explain what went wrong, and propose a fix.

Step 4: Approve or reject the fix. If the fix doesn't work, tell Claude what happened and iterate.

What makes a good error report

- The full error message — not just "it doesn't work"
- What you were doing when it happened
- What you expected to happen
- What actually happened instead
- What changed recently — "It worked until I added X"

Common debugging prompts

```
This test is failing. Run it, read the error, and fix it.

The page shows a blank screen. Check the console errors and trace the issue.

This function returns undefined instead of the user object. Why?

The CSS isn't loading. Check all the import paths.
```

Tip
When Claude fixes a bug, ask it to explain why it happened. Understanding the root cause helps you avoid similar bugs in the future. Try: "Fix it, and then explain what caused this so I can learn."

Writing tests with Claude Code

Testing isn't just for finding bugs — it's for preventing them. Claude Code is excellent at writing tests because it understands your code's logic.

Asking Claude to write tests

```
Write tests for the addTask function in src/utils/tasks.js

Create a test suite for the UserProfile component. Cover:
- renders with default props
- shows loading state
- handles error state
- displays user data correctly

Write integration tests for the /api/users endpoint
```

Test-Driven Development (TDD) with Claude

TDD means writing the test before writing the code. It's a powerful pattern that works brilliantly with Claude Code:

```
I want a function called calculateDiscount that:
- Takes a price and a discount percentage
- Returns the discounted price
- Throws an error if discount is over 100%

Write the tests FIRST, then implement the function to pass them.
```

Running tests

```
Run all tests and show me the results

Run only the tests that failed last time

Run tests in watch mode so they re-run on every file change
```

Tip

After Claude fixes a bug, always ask it to write a test for that specific bug. This prevents the same bug from coming back. Say: "Now write a test that would catch this bug if it ever happens again."

Try This

Intentionally break something in your project (delete a semicolon, misspell a variable name, remove an import). Then paste the resulting error to Claude Code and watch it find and fix the issue. Then ask Claude to write a test that prevents this bug.

Key Takeaways

- ✓ Always paste the FULL error message, not just “it doesn’t work”
- ✓ Tell Claude what you were doing, what you expected, and what actually happened
- ✓ Claude reads the relevant files, traces the cause, and proposes a fix
- ✓ Ask Claude to explain the root cause so you learn from each bug
- ✓ After fixing a bug, write a test to prevent it from coming back
- ✓ TDD with Claude: write tests first, then implement the code to pass them

Chapter 9

Git & Version Control

Commits, branches, and pull requests without leaving Claude Code

Git is a tool that tracks changes to your files over time. Think of it as “save points” for your project — you can always go back to a previous version. Claude Code handles git operations naturally through conversation.

Key concepts (simplified)

- Repository (repo): Your project folder, tracked by git
- Commit: A save point — a snapshot of all your files at a moment in time
- Branch: A parallel copy of your project where you can experiment safely
- Pull Request (PR): A request to merge changes from one branch into another
- Push: Upload your commits to a remote server (like GitHub)

Initialize git in your project

If your project isn’t tracked by git yet:

```
Initialize a git repository and create an initial commit
```

Making commits

After making changes, ask Claude to commit them:

```
Commit all changes with a descriptive message

Commit the changes to the login component. Use conventional commit format.
```

Working with branches

```
Create a new branch called feature/search-bar and switch to it

Switch back to the main branch

Merge feature/search-bar into main
```

Creating pull requests

```
Create a pull request for this branch. Include a summary of all
changes, what was added, and testing instructions.
```

Note

Claude Code can write excellent commit messages and PR descriptions because it has read all your changes. Let it handle the writing — you focus on reviewing.

Try This

Initialize a git repo in your project (if you haven't already). Make a change, then ask Claude to commit it. Create a new branch, make another change, and merge it back.

Key Takeaways

- ✓ Git tracks changes to your files — think "save points"
- ✓ Claude can initialize repos, commit, branch, merge, and create PRs
- ✓ Let Claude write commit messages and PR descriptions — it's read all your changes
- ✓ Use branches for experiments so you can safely try things
- ✓ Every project should use git from day one

Chapter 10

Understanding Tools

What Claude Code can do under the hood — and how permissions work

---

When you ask Claude Code to do something, it doesn't just generate text. It uses a set of built-in tools to interact with your computer. Understanding these tools helps you understand what Claude can and cannot do.

Claude Code's built-in tools

- **`Read`** — Read the contents of any file in your project
- **`Write`** — Create new files or completely replace file contents
- **`Edit`** — Make targeted edits to specific parts of a file (surgical changes)
- **`Bash`** — Run any terminal command — install packages, start servers, run scripts
- **`Glob`** — Search for files by name pattern (e.g., find all .tsx files)
- **`Grep`** — Search for text inside files (e.g., find every file that mentions "userAuth")
- **`Browser`** — Open URLs, take screenshots, interact with web pages (via Playwright)
- **`Subagent`** — Spawn a smaller Claude instance to handle a sub-task independently

How Claude chooses tools

You don't need to tell Claude which tool to use. When you say "create a new file called utils.js", Claude automatically picks the Write tool. When you say "find all TODO comments", it picks Grep. When you say "install express", it picks Bash and runs `npm install express`.

Sometimes Claude chains multiple tools together. For example, "find the bug in the login flow" might trigger: Glob (find relevant files) → Read (read them) → Edit (fix the bug) → Bash (run tests to verify).

The permission model

Claude Code asks for your permission before taking certain actions. There are three permission modes:

Normal (default)
Claude asks before running bash commands or editing files. You approve each action.

Auto-Accept
Claude runs without asking — faster but less control. Toggle with `Shift+Tab`.

Plan Mode
Claude can only read files and think — it cannot make any changes. Great for code review.

Allowed and blocked tools

You can configure which tools Claude is allowed to use in `settings.json` or per-project in CLAUDE.md. For example, you might block the Bash tool in a sensitive production repo to prevent Claude from running commands.

Tip

When you're learning, stick with Normal mode. Review every action Claude takes. Once you're comfortable and trust your setup, switch to Auto-Accept for speed.

Try This

Start Claude Code and ask: "What tools do you have available?" Then try `Shift+Tab` to cycle through permission modes. Watch how Claude's behavior changes in Plan Mode vs Normal.

Key Takeaways

- ✓ Claude Code has 8 built-in tools: Read, Write, Edit, Bash, Glob, Grep, Browser, Subagent
- ✓ Claude automatically picks the right tool based on your request
- ✓ Tools can be chained together for complex tasks
- ✓ Three permission modes: Normal (default), Auto-Accept, Plan Mode
- ✓ Shift+Tab cycles between permission modes during a session

Chapter 11

Building a Real Project

Step-by-step walkthrough: from an idea to a working app

Let’s put everything together. In this chapter, we’ll walk through building a complete project from start to finish using Claude Code. The project: a personal task manager web application.

Step 1: Plan before building

```
I want to build a personal task manager web app. Features:
- Add, edit, delete tasks
- Mark tasks as complete
- Filter by status (all, active, completed)
- Tasks persist in local storage

What tech stack do you recommend for a beginner? Give me a plan
before writing any code.
```

Claude will suggest a tech stack (likely vanilla HTML/CSS/JS for beginners or React for something more structured) and outline a plan. Review and approve before proceeding.

Step 2: Build the foundation

```
Great, let's go with that plan. Build step 1: the basic HTML
structure and CSS styling. Make it look clean and modern.
```

Step 3: Add features one at a time

Don’t ask for everything at once. Build incrementally:

```
Now add the ability to add new tasks. When I type in the
input and press Enter, it should appear in the list.

--- (after testing) ---

That works. Now add the ability to mark tasks as complete
by clicking on them. Completed tasks should be styled differently.

--- (after testing) ---

Now add the filter buttons: All, Active, Completed.
```

Step 4: Test and fix

```
Open index.html in the browser and test all features.
If something doesn't work, let me know what's wrong.
```

Step 5: Polish and commit

```
The app works! Now:
1. Add local storage so tasks survive page refresh
2. Add a nice empty state when there are no tasks
3. Commit everything with a good message
```

The build cycle

Notice the pattern: Plan → Build one feature → Test → Fix → Next feature → Polish → Commit. This is how professional developers work, just with Claude doing the heavy lifting.

Tip

After finishing a feature, run `/compact` before starting the next one. This keeps Claude focused and prevents it from getting confused by old context.

Try This

Build the task manager app from this chapter. Follow the steps one by one. When you're done, challenge yourself: add a new feature Claude didn't plan (like due dates or categories).

Key Takeaways

- ✓ Always start with a plan — ask Claude to outline before building
- ✓ Build one feature at a time, test it, then move to the next
- ✓ The cycle: Plan → Build → Test → Fix → Commit
- ✓ /compact between features to keep context clean
- ✓ Don't ask for everything at once — iterate incrementally

Chapter 12

CLAUDE.md & Configuration

Give Claude persistent memory and project-specific instructions

Every time you start a new Claude Code session, Claude starts fresh — it doesn't remember previous conversations. But there's a way to give it persistent context: the `CLAUDE.md` file.

What is CLAUDE.md?

It's a simple text file in your project root that Claude reads automatically every time it starts. Think of it as a "briefing document" for Claude — who you are, what this project is, what rules to follow, and any context Claude needs to be helpful.

Creating your CLAUDE.md

```
Create a CLAUDE.md file for this project. Include:
- What this project is
- The tech stack we're using
- Our coding conventions
- Any rules (like "always write tests" or "use TypeScript")
```

Example CLAUDE.md

```
# Task Manager App

## About
A personal task manager built with React + TypeScript.

## Tech Stack
- React 18 with hooks
- TypeScript strict mode
- Tailwind CSS for styling
- Vitest for testing

## Rules
- Always write tests for new features
- Use functional components, never class components
- Keep components under 100 lines
- Use conventional commit messages
```

Three levels of CLAUDE.md

Global — `~/.claude/CLAUDE.md`
Applied to ALL your projects. Put general preferences here (coding style, language, etc.)

Project — `your-project/CLAUDE.md`
Applied to this specific project. Tech stack, project rules, architecture decisions.

Folder — `your-project/src/CLAUDE.md`
Applied only when working in this folder. Component-specific guidelines.

The # shortcut

During a session, type # followed by text to instantly add a note to your CLAUDE.md. Useful when you discover a new rule mid-session:

```
# Always use date-fns for date formatting, never moment.js
```

settings.json

Claude Code also has a settings file at `~/.claude/settings.json` for things like model selection, permitted tools, and MCP servers. You'll rarely need to edit this manually — use `/config` instead.

Try This
Create a CLAUDE.md in your project root. Include the tech stack, 3 rules, and a note about the project's purpose. Start a new Claude Code session and notice how Claude references these instructions automatically.

Key Takeaways

- ✓ CLAUDE.md is Claude's "briefing document" — it reads it on every session start
- ✓ Three levels: global (~/.claude/), project root, and folder-level
- ✓ Include: project description, tech stack, coding rules, conventions
- ✓ Use # in a session to quickly add notes to CLAUDE.md
- ✓ settings.json controls model, tools, and MCP — use /config to edit

Chapter 13

Languages & Frameworks

How Claude Code works with Python, JavaScript, React, and more

Claude Code isn't limited to one programming language. It works with virtually any language or framework. But there are tips that make it work better with specific technologies.

Languages Claude Code excels at

- JavaScript / TypeScript: The strongest. Excellent with Node.js, React, Next.js, Express, and the entire JS ecosystem.
- Python: Very strong. Great with Flask, Django, FastAPI, data science (pandas, numpy), and scripting.
- HTML / CSS: Excellent for web design, responsive layouts, Tailwind CSS, and styling.
- Rust: Strong. Good with Cargo, error handling patterns, and systems programming.
- Go: Strong. Clean code generation, good with standard library patterns.
- SQL: Very good. Database queries, schema design, migrations.
- Shell / Bash: Excellent. Scripts, automation, system administration.

Framework-specific tips

React / Next.js

```
Create a Next.js app with App Router, TypeScript, and Tailwind.
Set up the project structure following Next.js best practices.

Create a reusable DataTable component with sorting, filtering,
and pagination. Use TypeScript generics so it works with any data type.
```

Python / FastAPI

```
Create a FastAPI server with:
- /users endpoint (CRUD)
- SQLAlchemy models
- Pydantic schemas for validation
- JWT authentication

Follow FastAPI best practices and add docstrings for auto-docs.
```

Working with existing codebases

When you open Claude Code in a project that already has code, Claude reads the project structure and tries to match the existing patterns. To help it:

- Document your tech stack in CLAUDE.md (Chapter 11)
- Include coding conventions (naming, file structure, import style)
- Mention the testing framework you use
- List any libraries Claude should or shouldn't use

Note
Claude Code adapts to your existing code style. If your project uses semicolons, Claude will use semicolons. If it uses tabs, Claude uses tabs. It mirrors your conventions.

Try This
Pick a language or framework you're interested in. Ask Claude to create a small starter project with best practices. Compare what Claude generates with official documentation — you'll learn both the language and how Claude approaches it.

Key Takeaways

- ✓ Claude Code works with any language — JS/TS and Python are the strongest
- ✓ Include your tech stack and conventions in CLAUDE.md for best results
- ✓ Claude mirrors your existing code style (semicolons, tabs, naming)
- ✓ Framework-specific prompts work best when you mention the framework by name
- ✓ Claude can set up entire project scaffolds following best practices

Chapter 14

Skills & Hooks

Extend Claude's abilities with reusable instruction sets and automations

Skills and hooks are two ways to customize Claude Code's behavior beyond CLAUDE.md. Skills give Claude new capabilities; hooks automate actions.

What are Skills?

A skill is a folder containing a `SKILL.md` file with structured instructions. When Claude encounters a task matching the skill's description, it automatically follows those instructions. Think of skills as "recipes" Claude can follow.

Example: a simple skill

```
# .claude/skills/create-component/SKILL.md

---
name: react-component-creator
description: Creates React components. Use when user says
"create component", "new component", "make a component".
---

## Instructions
1. Ask for: component name, props, purpose
2. Create in src/components/[Name]/
3. Include: component file, test file, index.ts export
4. Use TypeScript, functional component, hooks only
```

Finding skills

You don't have to write skills from scratch. Visit `skills.sh` — the official Claude skills platform — to browse and install community-made skills.

What are Hooks?

Hooks are scripts that run automatically before or after Claude's actions. They're defined in `settings.json` and can automate things like:

- Running a linter after every file edit
- Auto-formatting code before commits
- Sending notifications when Claude finishes a task
- Blocking certain file modifications

Note
Hooks are more advanced and require some scripting knowledge. Start with skills first — they're simpler and cover most use cases.

Try This

Visit `skills.sh` and browse the available skills. Install one that's relevant to your project. Then try creating your own simple skill following the example above.

Key Takeaways

- ✓ Skills are structured instruction files (SKILL.md) that give Claude new capabilities
- ✓ skills.sh is the official platform for browsing and sharing skills
- ✓ Hooks are automated scripts that run before/after Claude's actions
- ✓ Start with skills — they're simpler. Add hooks when you need automation.
- ✓ Skills live in .claude/skills/ in your project or globally

Chapter 15

MCP Servers

Connect Claude to GitHub, databases, Slack, and more

MCP (Model Context Protocol) is how Claude Code connects to external tools and services. Think of MCP servers as “plugins” that give Claude access to things outside your project folder.

What can MCP servers do?

- GitHub — read issues, create PRs, review code, manage repos
- Slack — read messages, post updates, search channels
- Databases — query data, run SQL, inspect schemas
- Playwright — browse websites, take screenshots, test web apps
- Google Workspace — read/write docs, spreadsheets, emails
- File system — access files outside your project folder

How to set up an MCP server

MCP servers are configured in your `settings.json` or through Claude’s settings UI. Here’s the general process:

1. Find the MCP server you need (often on GitHub or npm)

2. Add it to your configuration:

```
// In ~/.claude/settings.json
{
"mcpServers": {
"github": {
"command": "npx",
"args": ["-y", "@modelcontextprotocol/server-github"],
"env": {
"GITHUB_TOKEN": "your-token-here"
}
}
}
}
```

3. Restart Claude Code — it will connect to the server automatically.

Using MCP in conversation

Once connected, you can use it naturally:

```
Show me all open issues in our GitHub repo

Create a new issue: "Fix mobile layout on checkout page"

Read the last 10 messages in #engineering Slack channel
```

Warning

MCP servers can access external services and data. Only connect servers you trust, and be careful with API tokens. Never commit tokens to git.

Try This

If you have a GitHub account, set up the GitHub MCP server. Then try asking Claude to list your repositories or create a test issue. If you don't have GitHub, just read through this chapter — you'll use MCP later in your journey.

Key Takeaways

- ✓ MCP = Model Context Protocol — plugins that connect Claude to external services
- ✓ Common MCP servers: GitHub, Slack, databases, Playwright, Google Workspace
- ✓ Configured in settings.json with server name, command, and environment variables
- ✓ Use naturally in conversation — Claude handles the API calls
- ✓ Be careful with API tokens — never commit them to git

Chapter 16

API Integration

Connecting to external services, managing secrets, and building integrations

Most real applications need to talk to external services — payment processors, email providers, weather APIs, databases. Claude Code can build these integrations for you.

How to ask Claude to integrate an API

```
Integrate the Stripe API for processing payments.
I need:
- A checkout endpoint that creates a Stripe session
- A webhook handler for payment confirmation
- Error handling for failed payments

Use the official Stripe Node.js SDK. My Stripe secret key
is in the STRIPE_SECRET_KEY environment variable.
```

Environment variables & secrets

API keys and passwords should never be written directly in your code. Instead, use environment variables:

```
# Create a .env file (never commit this to git!)
STRIPE_SECRET_KEY=sk_test_your_key_here
DATABASE_URL=postgresql://user:pass@localhost/mydb
SENDGRID_API_KEY=SG.your_key_here
```

```
# In your code, access with:
process.env.STRIPE_SECRET_KEY // JavaScript
os.environ['DATABASE_URL'] # Python
```

Warning
Always add `.env` to your `.gitignore` file. If you accidentally commit secrets to git, consider them compromised — rotate them immediately.

Common API patterns

```
Set up a REST API client for the OpenWeather API.
Create helper functions for: get current weather, get forecast.
Add caching so we don't hit rate limits.
Handle errors gracefully (network failures, invalid API key).
```

Working with databases

Claude can set up database connections, write queries, create migrations, and design schemas:

```
Set up a SQLite database for our task manager.
Create tables for users, tasks, and categories.
Write functions for CRUD operations.
Add a migration script that creates the initial schema.
```

Try This

Pick a free public API (like OpenWeather, JSONPlaceholder, or PokéAPI). Ask Claude to build a small app that fetches and displays data from it. Practice using environment variables for the API key.

Key Takeaways

- ✓ Claude can integrate any API — describe what you need and which SDK to use
- ✓ NEVER put API keys directly in code — use environment variables (.env files)
- ✓ Always add .env to .gitignore
- ✓ Ask Claude to add error handling and caching for API calls
- ✓ Claude can set up databases, write queries, and create migrations

Chapter 17

Production Deployment

Going live — deploy your project to the internet

You've built something. Now it's time to put it on the internet so others can use it. Claude Code can help you deploy to popular platforms.

Deployment platforms for beginners

- Vercel: Best for: React, Next.js, static sites. Free tier available. Deploys from GitHub.
- Netlify: Best for: Static sites, JAMstack. Free tier. Drag-and-drop or GitHub deploy.
- Railway: Best for: Full-stack apps with databases. Simple pricing. Supports Node, Python, Go.
- Render: Best for: APIs, web services, databases. Free tier for static sites.
- GitHub Pages: Best for: Simple static HTML/CSS sites. Completely free.

Deploying with Claude Code

Ask Claude to prepare your project for deployment:

```
Prepare this project for deployment to Vercel.
- Make sure the build command works
- Set up environment variables correctly
- Create a vercel.json if needed
- Walk me through the deployment steps
```

The deployment checklist

- Environment variables — set them in the platform's dashboard, not in code
- Build command — verify `npm run build` (or equivalent) works locally
- Git — commit all changes before deploying
- .gitignore — make sure node_modules, .env, and build artifacts are excluded
- README — document how to run the project locally
- Error pages — add a custom 404 page

Continuous deployment (CI/CD)

Most platforms support continuous deployment: every time you push code to GitHub, it automatically rebuilds and deploys. Claude can set this up:

```
Set up a GitHub Actions workflow that:
1. Runs tests on every pull request
2. Deploys to Vercel on merge to main
3. Sends a Slack notification on success or failure
```

Tip

Always deploy early, even if your project isn't "finished." Having a live URL you can share motivates you to keep building, and you catch deployment issues early.

Try This

Deploy your project (even if it's simple) to Vercel or GitHub Pages. Ask Claude to walk you through each step. Share the live URL with a friend!

Key Takeaways

- ✓ Vercel and Netlify are the easiest for beginners — free tiers, GitHub integration
- ✓ Always set environment variables in the platform's dashboard, never in code
- ✓ Verify your build works locally before deploying
- ✓ Continuous deployment auto-deploys on every git push — Claude can set this up
- ✓ Deploy early and often — don't wait for "perfect"

Chapter 18

Agents, Best Practices & Next Steps

Background agents, daily workflow, cost management, and where to go from here

---

Claude Code can run as an agent — working on tasks in the background while you do other things, or spawning subagents to handle parts of a larger task in parallel.

Background agents

A background agent works on a task while you continue doing other things. It's like giving Claude a task and saying "work on this, I'll check back later."

```
claude --background "Refactor all the API routes to use async/await
and add error handling. Commit when done."
```

Claude works on the task in the background. You can check progress or continue working on other things.

Subagents

For complex tasks, Claude can spawn subagents — smaller Claude instances that each handle a piece of the work. For example, when building a full feature, Claude might:

- Spawn a subagent to build the UI components
- Spawn another to write the API endpoints
- Spawn a third to write the tests
- Coordinate the results itself

You don't need to manage subagents manually — Claude decides when to use them based on the complexity of your request.

When to use agents

- Large refactors — changing patterns across many files
- Test writing — generating tests for existing code
- Code migrations — upgrading a library across the codebase
- Documentation — generating docs for all your functions

Note
Agents and subagents use more API tokens than regular conversations because they're doing more work. Keep an eye on costs with `/cost`.

Try This
Try running a background agent on a simple task: `claude --background "Add JSDoc comments to all exported functions in src/"`. Check the results when it's done.

Key Takeaways

- ✓ Background agents work on tasks while you do other things
- ✓ Subagents are smaller Claude instances handling parts of a larger task
- ✓ Claude decides when to use subagents based on task complexity
- ✓ Great for large refactors, test writing, migrations, and documentation
- ✓ Agents use more tokens — monitor costs with /cost

The daily workflow

Here's a recommended workflow for daily use:

1. Open your project — `cd your-project && claude`
2. Check status — `/status` to verify connection and model
3. State your goal — Tell Claude what you're working on today — one clear task
4. Build incrementally — One feature at a time. Test after each change.
5. /compact periodically — Every 15–20 minutes to keep context clean
6. Commit frequently — Ask Claude to commit after each working feature
7. Review costs — `/cost` at the end of the session
8. Update CLAUDE.md — Add any new rules or conventions you discovered
9. Exit cleanly — `exit` or Ctrl+D when done

Cost management

Claude Code uses API tokens, and costs can add up with heavy use. Here's how to stay in control:

- Use `/cost` to check your spending during sessions
- Use `/compact` to reduce token usage in long sessions
- Use `-p` for quick questions (cheaper than full sessions)
- Be specific in prompts — vague prompts cause Claude to explore more (more tokens)
- One task per session — don't keep stale context around

Common mistakes to avoid

- Vague prompts → Be specific. Use WHAT/WHERE/HOW/VERIFY.
- Ignoring diffs → Always review what Claude changes. Don't blindly accept.
- No CLAUDE.md → Create one on day one. It makes every session better.
- Not using git → Always commit before major changes. It's your safety net.
- Too many tasks per session → One task at a time. /clear between unrelated tasks.
- Never running /compact → Context degrades over long sessions. Compact regularly.

Where to go from here

You now have a solid foundation. Here are the best resources to keep learning:

- Official docs: `code.claude.com` — the authoritative reference
- Skills platform: `skills.sh` — browse and install community skills
- Community: `r/ClaudeCode` on Reddit — tips, workflows, and discussions
- awesome-claude-code on GitHub — a curated list of tools, extensions, and resources
- Claude Code Discord — join the Claude Developers Discord for help and sharing

Chapter A

Quick Reference

Essential commands, shortcuts, and prompts on one page

Installation

```
# Mac / Linux
curl -fsSL https://claude.ai/install.sh | bash

# Windows (PowerShell)
irm https://claude.ai/install.ps1 | iex

# Verify
claude --version
```

Essential slash commands

`/help` Show all commands

`/status` Connection, model, session info

`/compact` Compress conversation (save context)

`/clear` Wipe conversation completely

`/cost` Show session cost

`/config` Open settings

`/doctor` Auto-diagnose problems

`/bug` Report a bug to Anthropic

Keyboard shortcuts

`Esc Esc` Undo last Claude action

`Tab` Toggle extended thinking

`Shift+Tab` Cycle permission modes

`Ctrl+R` Search prompt history

`Ctrl+C` Cancel current response

`# text` Quick-add to CLAUDE.md

Command-line flags

`-p "prompt"` — Run one-off prompt, print result, exit

`--background "task"` — Run task in background

`--permission-mode plan` — Start in read-only Plan Mode

The prompt formula

WHAT do you want? + WHERE in the project? + HOW (constraints)? + VERIFY (how to test)?

Daily workflow

```
cd project → claude → /status → state goal → build → /compact → commit → /cost → exit
```

Chapter B

Troubleshooting

Common problems and how to fix them

"command not found: claude"
Close your terminal and open a new one. The installer adds Claude to your PATH but it only takes effect in new sessions. If it persists, re-run the installer.

"Authentication failed" or browser doesn't open
Run `claude logout` then `claude` again to re-authenticate. Make sure you have an active paid plan (Pro or Max).

Claude seems to forget things mid-session
Your context window is full. Run `/compact` to compress the conversation. For completely unrelated tasks, start a new session.

Claude makes changes I didn't want
Press Esc Esc to rewind the last action. Always review diffs before approving. Use Plan Mode (`Shift+Tab`) for read-only exploration.

Claude is slow or times out
Complex tasks take longer. If it's consistently slow, check `/status` for connection issues. Run `/doctor` to auto-diagnose.

Permission errors during installation
Never use `sudo` with npm install. Use `nvm` to manage Node.js, or use the native installer which has zero dependencies.

MCP server won't connect
Check that the server is correctly configured in `settings.json`. Verify API tokens are valid. Restart Claude Code after config changes.

Claude edits the wrong file
Be specific. Use `@filename` to point Claude to the exact file. Include the full path if there are similarly named files.

Git errors or merge conflicts
Ask Claude: "Show me the current git status and help me resolve any conflicts." Claude can read the conflict markers and propose resolutions.

High costs / unexpected spending
Use `/cost` regularly. Use `/compact` to reduce token usage. Use `-p` for quick one-off questions. Be specific in prompts.

Still stuck?

- Run `/doctor` — it auto-detects most issues
- Check the official docs: `code.claude.com`
- Ask the community: `r/ClaudeCode` on Reddit
- Report bugs: use `/bug` inside Claude Code

Chapter C

Challenge Exercises

Progressive challenges to practice what you've learned

Each challenge builds on skills from the chapters. Try them yourself first, then use Claude Code to help if you get stuck.

Beginner Challenges

Challenge 1: Personal Bio Page
Create a single HTML page with your name, photo placeholder, bio, and 3 links. Style it with CSS. (Ch 3)

Challenge 2: File Organizer
Ask Claude to create a bash script that sorts files in a folder by extension into subfolders. (Ch 5, 10)

Challenge 3: Git First Steps
Initialize a git repo, make 3 commits with good messages, create a branch, and merge it. (Ch 9)

Challenge 4: Prompt Challenge
Ask Claude the same question 3 ways: vague, medium, and using WHAT/WHERE/HOW/VERIFY. Compare outputs. (Ch 4)

Intermediate Challenges

Challenge 1: Task Manager App
Build a full task manager with add/edit/delete/filter/local storage. Deploy it. (Ch 10, 17)

Challenge 2: API Weather App
Build a weather dashboard that fetches data from a public API and displays a 5-day forecast. (Ch 16)

Challenge 3: Test Suite
Take any project and ask Claude to write a comprehensive test suite. Aim for 80%+ coverage. (Ch 8)

Challenge 4: CLAUDE.md Master
Create a detailed CLAUDE.md for a real project with tech stack, rules, conventions, and forbidden patterns. (Ch 11)

Advanced Challenges

Challenge 1: Full-Stack App
Build a blog with user auth, a database, CRUD posts, and deploy to Railway. (Ch 10, 13, 16, 17)

Challenge 2: MCP Integration
Set up the GitHub MCP server and ask Claude to create issues, list PRs, and post comments. (Ch 14)

Challenge 3: Multi-Agent Refactor
Use background agents to refactor a project: rename variables, add types, write docs — all in parallel. (Ch 15)

Challenge 4: CI/CD Pipeline
Set up GitHub Actions that run tests, lint, and deploy on every push to main. (Ch 17)

You're ready.

The best way to learn Claude Code is to use it every day. Start small, be specific, review every change, and build up from there. You've got this.

Happy building!

by AI Agents Accelerator